

Управление состоянием в React на основе Signals



Какие средства есть и их недостатки?

- Встроенные useState
 - для каждого значения нужен свой useState (можно решить через useReducer, но становится много кода)
 - Проброс пропсов (prop drilling) (можно решить через Context)
 - рендер всех дочерних элементов, при изменении значения
- Redux
 - много бойлерплейт кода (action, selector, reducer)
 - иммутабельность стейта (решается через immer в RTK)
 - рендер всех дочерних элементов, при изменении стейта
- Mobx подобные
 - магия под капотом
 - сложность отладки

Signals

Signal — это **реактивный** примитив, который хранит значение и автоматически отслеживает, кто его читает.

Ключевые характеристики:

- Контейнер значения с функциями **чтения** и **записи**.
- **Автоматическое отслеживание зависимостей**: когда значение сигнала меняется, обновляются только те компоненты (или части компонентов), которые его читают.
- **Ленивость по умолчанию**: сигналы, на которые никто не подписан, не вызывают обновлений.
- **Прямой доступ**: не нужны селекторы или wrapper-функции — сигнал читается напрямую, и компонент автоматически подписывается на его изменения.

Как работают Signals: базовая модель

```
import { signal, computed, effect } from '@preact/signals-react'

const count : Signal<number> = signal( value: 0)

console.log(count.value) // 0
count.value = 5 // уведомляет подписчиков

const double : ReadonlySignal<number> = computed(() => count.value * 2)

effect(() => {
  console.log( ...data: `Значение: ${count.value}, удвоенное: ${double.value}`)
})
```

Интеграция Signals с React: @preact/signals-react

- `npm install @preact/signals-react`
- `@preact/signals-react-transform`

```
import react from '@vitejs/plugin-react'
import babel from '@rolldown/plugin-babel';
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), babel({
    plugins: [['module:@preact/signals-react-transform']],
    // Ensure the transform runs before Oxc's JSX transformation
  })],
})
```

А как же асинхронные запросы?

- В эффekte запрашивать данные, сохранять в сигналы
- Использовать [@reduxjs/toolkit/query](#), [@tanstack/react-query](#) (на мой взгляд более предпочтительно)

Итоги

- **Легко тестируется**, если обернуть все в класс
- Реальная **реактивность** в React :D
- **Производительность** (нет перерендеров дочерних элементов)
- **Удобство** — сеттер и геттер в одном объекте
- **Простота подключения** и использования, не нужны провайдеры, обертки и т.д.
- Использовать или нет в проде дело каждого, но **попробовать обязательно** нужно